Scenario Based Interview

Pyspark vs *Spark SQL*



Ganesh. R

Swipe Pages

#Problem Statement You are the restaurant owner and you want to analyze a possible expansion (there will be at least one customer every day).

Compute the moving average of how much the customer paid in a seven days window (i.e., current day + 6 days before). average_amount should be rounded to two decimal places.

Return the result table ordered by visited_on in ascending order.

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.functions import col, sum, round, window
from pyspark.sql.types import DateType

# Initialize Spark session
spark = SparkSession.builder.appName("MovingAverage").getOrCreate()

# Sample data
data = [
    (1, "Jhon", "2019-01-01", 100),
    (2, "Daniel", "2019-01-02", 110),
    (3, "Jade", "2019-01-03", 120),
    (4, "Khaled", "2019-01-04", 130),
    (5, "Winston", "2019-01-05", 110),
    (6, "Elvis", "2019-01-06", 140),
    (7, "Anna", "2019-01-07", 150),
    (8, "Maria", "2019-01-08", 80),
    (9, "Jaze", "2019-01-09", 110),
    (1, "Jhon", "2019-01-10", 130),
    (3, "Jade", "2019-01-10", 150),
]

# Create DataFrame
columns = ["customer_id", "name", "visited_on", "amount"]
df = spark.createDataFrame(data, schema=columns)

df.display()
df.printSchema()
```

```
root
 |-- customer_id: long (nullable = true)
 |-- name: string (nullable = true)
 |-- visited_on: string (nullable = true)
 |-- amount: long (nullable = true)
```

```
# Define a window specification
window_spec = Window.orderBy("visited_on").rowsBetween(-6, 0)

# Calculate the rolling sum and average
result_df = (
```

```
    df.groupBy("visited_on")
    .agg(sum("amount").alias("daily_amount"))
    .withColumn("amount", sum("daily_amount").over(window_spec))
    .withColumn("average_amount",
round(avg("daily_amount").over(window_spec), 2))
)

# Filter to include only rows where row_number >= 7
result_df = (
    result_df.withColumn("row_number",
row_number().over(Window.orderBy("visited_on")))
    .filter(col("row_number") >= 7)
    .select("visited_on", "amount", "average_amount")
)

# Show the result
result_df.display()

df.createOrReplaceTempView("Customer")

%sql
WITH CustomerGrouped AS (
  SELECT
    visited_on,
    SUM(amount) AS total_amount
  FROM
    Customer
  GROUP BY
    visited_on
),
MovingAverage AS (
  SELECT
    visited_on,
    total_amount,
    SUM(total_amount) OVER (
      ORDER BY
        visited_on ROWS BETWEEN 6 PRECEDING
        AND CURRENT ROW
    ) AS sum_amount_7d
  FROM
    CustomerGrouped
)
SELECT
  visited_on,
  sum_amount_7d AS amount,
  ROUND(sum_amount_7d / 7, 2) AS average_amount
FROM
  MovingAverage
WHERE
  DATEDIFF(
```

```
    visited_on,
    (
      SELECT
        MIN(visited_on)
      FROM
        CustomerGrouped
    )
  ) >= 6
ORDER BY
  visited_on;
```

#Problem Statement Write a solution to find the students who meet the following criteria:

Have taken all mandatory courses and at least two elective courses offered in their major. Achieved a grade of A in all mandatory courses and at least B in elective courses. Maintained an average GPA of at least 2.5 across all their courses (including those outside their major). Return the result table ordered by student_id in ascending order.

###PySpark

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, sum, when

# Initialize Spark session
spark = SparkSession.builder \
    .appName("University Data Analysis") \
    .getOrCreate()

# Sample data for students table
students_data = [
    (1, 'Alice', 'Computer Science'),
    (2, 'Bob', 'Computer Science'),
    (3, 'Charlie', 'Mathematics'),
    (4, 'David', 'Mathematics')
]

# Sample data for courses table
courses_data = [
    (101, 'Algorithms', 3, 'Computer Science', 'yes'),
    (102, 'Data Structures', 3, 'Computer Science', 'yes'),
    (103, 'Calculus', 4, 'Mathematics', 'yes'),
    (104, 'Linear Algebra', 4, 'Mathematics', 'yes'),
    (105, 'Machine Learning', 3, 'Computer Science', 'no'),
    (106, 'Probability', 3, 'Mathematics', 'no'),
    (107, 'Operating Systems', 3, 'Computer Science', 'no'),
    (108, 'Statistics', 3, 'Mathematics', 'no')
]

# Sample data for enrollments table
enrollments_data = [
    (1, 101, 'Fall 2023', 'A', 4.0),
    (1, 102, 'Spring 2023', 'A', 4.0),
    (1, 105, 'Spring 2023', 'A', 4.0),
    (1, 107, 'Fall 2023', 'B', 3.5),
    (2, 101, 'Fall 2023', 'A', 4.0),
    (2, 102, 'Spring 2023', 'B', 3.0),
    (3, 103, 'Fall 2023', 'A', 4.0),
    (3, 104, 'Spring 2023', 'A', 4.0),
    (3, 106, 'Spring 2023', 'A', 4.0),
    (3, 108, 'Fall 2023', 'B', 3.5),
    (4, 103, 'Fall 2023', 'B', 3.0),
```

```
    (4, 104, 'Spring 2023', 'B', 3.0)
]

# Create DataFrames
students_df = spark.createDataFrame(students_data, ["student_id",
"name", "major"])
courses_df = spark.createDataFrame(courses_data, ["course_id", "name",
"credits", "major", "mandatory"])
enrollments_df = spark.createDataFrame(enrollments_data,
["student_id", "course_id", "semester", "grade", "GPA"])

students_df.display()
courses_df.display()
enrollments_df.display()


# Filter students with AVG GPA >= 2.5
filtered_students =
enrollments_df.groupBy("student_id").agg(avg("GPA").alias("avg_GPA")).
filter(col("avg_GPA") >= 2.5)
# Join DataFrames
student_courses = students_df.join(courses_df, students_df.major ==
courses_df.major) \
                             .join(filtered_students, "student_id") \
                             .join(enrollments_df, ["student_id",
"course_id"], "left")

# Apply conditions
result_df = student_courses.groupBy("student_id").agg(
    sum(when((col("mandatory") == 'yes') & (col("grade") == 'A'),
1).otherwise(0)).alias("mandatory_A_count"),
    sum(when(col("mandatory") == 'yes',
1).otherwise(0)).alias("mandatory_count"),
    sum(when((col("mandatory") == 'no') & (col("grade").isNotNull()),
1).otherwise(0)).alias("non_mandatory_count"),
    sum(when((col("mandatory") == 'no') & (col("grade").isin("A",
"B")), 1).otherwise(0)).alias("non_mandatory_AB_count")
).filter(
    (col("mandatory_A_count") == col("mandatory_count")) &
    (col("non_mandatory_count") >= 2) &
    (col("non_mandatory_count") == col("non_mandatory_AB_count"))
).select("student_id").orderBy("student_id")
result_df.display()

students_df.createOrReplaceTempView("students")
courses_df.createOrReplaceTempView("courses")
enrollments_df.createOrReplaceTempView("enrollments")
```

###Spark SQL

```
%sql
WITH T AS (
    SELECT e.student_id
    FROM enrollments e
    GROUP BY e.student_id
    HAVING AVG(e.GPA) >= 2.5
)
SELECT t.student_id
FROM T t
JOIN students s ON t.student_id = s.student_id
JOIN courses c ON s.major = c.major
LEFT JOIN enrollments e ON t.student_id = e.student_id AND c.course_id
= e.course_id
GROUP BY t.student_id
HAVING
    SUM(CASE WHEN c.mandatory = 'yes' THEN (CASE WHEN e.grade = 'A'
THEN 1 ELSE 0 END) ELSE 0 END) =
    SUM(CASE WHEN c.mandatory = 'yes' THEN 1 ELSE 0 END)
    AND SUM(CASE WHEN c.mandatory = 'no' AND e.grade IS NOT NULL THEN
1 ELSE 0 END) =
    SUM(CASE WHEN c.mandatory = 'no' THEN (CASE WHEN e.grade IN ('A',
'B') THEN 1 ELSE 0 END) ELSE 0 END)
    AND SUM(CASE WHEN c.mandatory = 'no' AND e.grade IS NOT NULL THEN
1 ELSE 0 END) >= 2
ORDER BY t.student_id;
```

Problem Statement



Write an SQL query to retrieve a list of department names along with a comma-separated list of employee names in each department. The result should include:

- deptname - The name of the department.
- name - A comma-separated list of employee names who belong to that department.

Input Table Data

```
# Define schema for dept table
emp_schema = StructType([
StructField("empid", IntegerType(), False), StructField("empname", StringType(), False),
StructField("salary", IntegerType(), False), StructField("deptid", IntegerType(), False) ])

# Define schema for dept table
dept_schema = StructType([ StructField("deptid", IntegerType(), False),
  StructField("deptname", StringType(), False) ])

# Create DataFrame for emp table
emp_data = [
  (1, 'Nikitha', 45000, 206), (2, 'Ashish', 42000, 207), (3, 'David', 40000, 206),
  (4, 'Ram', 50000, 207), (5, 'John', 35000, 208), (6, 'Mark', 50000, 207),
  (7, 'Aravind', 39000, 208)]
emp_df = spark.createDataFrame(emp_data, schema=emp_schema)

# Create DataFrame for dept table
dept_data = [(206, 'HR'), (207, 'IT'), (208, 'Finance')]
dept_df = spark.createDataFrame(dept_data, schema=dept_schema)

# Show the DataFrames
emp_df.display()
dept_df.display()
```

Expected Output

```
Deptname | Name
--------------------
Finance     | Aravind,John
HR          |  Nikitha,David
IT          | Ram,Mark
```

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, IntegerType,
StringType
from pyspark.sql import functions as F
from pyspark.sql.window import Window

# Initialize SparkSession
spark = SparkSession.builder.appName("example").getOrCreate()

# Define schema for emp table
emp_schema = StructType([
    StructField("empid", IntegerType(), False),
    StructField("empname", StringType(), False),
    StructField("salary", IntegerType(), False),
    StructField("deptid", IntegerType(), False)
])

# Define schema for dept table
dept_schema = StructType([
    StructField("deptid", IntegerType(), False),
    StructField("deptname", StringType(), False)
])

# Create DataFrame for emp table
emp_data = [
    (1, 'Nikitha', 45000, 206),
    (2, 'Ashish', 42000, 207),
    (3, 'David', 40000, 206),
    (4, 'Ram', 50000, 207),
    (5, 'John', 35000, 208),
    (6, 'Mark', 50000, 207),
    (7, 'Aravind', 39000, 208)
]
emp_df = spark.createDataFrame(emp_data, schema=emp_schema)

# Create DataFrame for dept table
dept_data = [
    (206, 'HR'),
    (207, 'IT'),
    (208, 'Finance')
]
dept_df = spark.createDataFrame(dept_data, schema=dept_schema)

# Show the DataFrames
emp_df.display()
dept_df.display()

dept_df.createOrReplaceTempView('emp')
emp_df.createOrReplaceTempView('dept')
```

```
sql_query = """
WITH cte AS (
    SELECT empname, deptname,
           RANK() OVER (PARTITION BY deptname ORDER BY salary DESC) AS
rnk
    FROM emp
    INNER JOIN dept
    USING (deptid)
)
SELECT deptname,
       CONCAT_WS(',', COLLECT_LIST(empname)) AS name
FROM cte
WHERE rnk = 1
GROUP BY deptname
"""

# Execute the SQL query
result_df = spark.sql(sql_query)

# Show the result
result_df.display()

# Join the DataFrames
joined_df = emp_df.join(dept_df, on="deptid")

# Define the Window specification for ranking
window_spec = Window.partitionBy("deptname").orderBy(F.desc("salary"))

# Add rank column using the Window specification
ranked_df = joined_df.withColumn("rnk", F.rank().over(window_spec))

# Filter rows where rank is 1 (highest salary)
top_salaries_df = ranked_df.filter(F.col("rnk") == 1)

# Group by department and aggregate employee names
result_df = top_salaries_df.groupBy("deptname").agg(
    F.concat_ws(",", F.collect_list("empname")).alias("name")
)

# Show the result
result_df.display()
```

Problem Statement



DataFrame containing employee information across different departments. Each record includes an employee ID (eid), department name (dept), and a score (scores). The goal is to perform the following operations:

1. Identify Top Scorers: For each department, identify the employee(s) with the highest score. If there are multiple employees with the same highest score in a department, they should all be considered top scorers.
2. Right Join with All Employees: Combine the list of all employees (eid, dept) with the top scorers, ensuring that every employee is present in the final result, even if they are not a top scorer in their department. The scores for non-top scorers should be null.
3. Order the Results: The final output should be sorted by the employee ID (eid).

Input Table Data

```
schema = StructType([
  StructField("eid", IntegerType(), True),
  StructField("dept", StringType(), True),
  StructField("scores", FloatType(), True)
])

# step 4: Create DataFrame with the specified data
data = [
  (1, 'd1', 1.0),
  (2, 'd1', 5.28),
  (3, 'd1', 4.0),
  (4, 'd2', 8.0),
  (5, 'd1', 2.5),
  (6, 'd2', 7.0),
  (7, 'd3', 9.0),
  (8, 'd4', 10.2)
]

df = spark.createDataFrame(data, schema)
```

Expected Output

eid	dept	scores
1	d1	5.28
2	d1	5.28
3	d1	5.28
4	d2	8
5	d1	5.28
6	d2	8
7	d3	9
8	d4	10.19

#Problem Statement:

###Employee Department Scores: I am giving a dataset containing employee information across different departments. Each record includes an employee ID (eid), department name (dept), and a score (scores). The goal is to perform the following operations:

###Identify Top Scorers: For each department, identify the employee(s) with the highest score. If there are multiple employees with the same highest score in a department, they should all be considered top scorers.

###Right Join with All Employees: Combine the list of all employees (eid, dept) with the top scorers, ensuring that every employee is present in the final result, even if they are not a top scorer in their department. The scores for non-top scorers should be null.

###Order the Results: The final output should be sorted by the employee ID (eid).

```
# step 1: Import necessary PySpark libraries
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, IntegerType,
StringType, FloatType
from pyspark.sql import functions as F
from pyspark.sql.window import Window

# step 2: Initialize Spark session
spark = SparkSession.builder \
    .appName("Create and Insert Table") \
    .getOrCreate()

# step 3: Define schema for the table
schema = StructType([
    StructField("eid", IntegerType(), True),
    StructField("dept", StringType(), True),
    StructField("scores", FloatType(), True)
])

# step 4: Create DataFrame with the specified data
data = [
    (1, 'd1', 1.0),
    (2, 'd1', 5.28),
    (3, 'd1', 4.0),
    (4, 'd2', 8.0),
    (5, 'd1', 2.5),
    (6, 'd2', 7.0),
    (7, 'd3', 9.0),
    (8, 'd4', 10.2)
]

df = spark.createDataFrame(data, schema)

# step 5: Save DataFrame as a table in the default database (or
```

```
specific database)
df.createOrReplaceTempView("empdept_tbl")

# step 6: Display the DataFrame content (optional)
df.display()
```

#Spark SQL

```
%sql
with t1 as(
  select
    *,
    dense_rank() over(
      partition by dept
      order by
        scores desc
    ) dn
  from
    empdept_tbl qualify dn = 1
),
t2 as(
  select
    eid,
    dept
  from
    empdept_tbl
)
select
  t2.eid,
  t2.dept,
  round(t1.scores, 2) as scores
from
  t1
  right join t2 on t1.dept = t2.dept
order by
  1;
```

#PySpark

```
# Step 1: Create window specification for dense_rank
window_spec = Window.partitionBy("dept").orderBy(F.desc("scores"))

# Step 2: Apply dense_rank and filter for rank = 1 (simulate QUALIFY
clause)
df_with_rank = df.withColumn("dn",
F.dense_rank().over(window_spec)).filter(F.col("dn") == 1)

# Step 3: Round the scores to the nearest integer
df_with_rank = df_with_rank.withColumn("scores",
```

```
F.round(F.col("scores"), 1))

# Step 4: Create t1 as the result of the above operation
t1 = df_with_rank.select("dept", "scores")

# Step 5: Create t2 as a selection of eid and dept
t2 = df.select("eid", "dept")

# Step 6: Perform right join between t1 and t2
result_df = t2.join(t1, on="dept", how="right")

# Step 7: Order by eid
result_df = result_df.orderBy("eid")
# display the final result
result_df.display()
```

#Explanation:

###Window Function: We use Window.partitionBy("dept").orderBy(F.desc("scores")) to define the partitioning and ordering for the DENSE_RANK window function.

###Filtering: Since PySpark doesn't support the QUALIFY clause, we filter the result after calculating the rank using .filter(F.col("dn") == 1).

###Join Operation: We then perform a right join between the t1 and t2 DataFrames based on the dept column.

###Ordering: Finally, the result is ordered by eid.

This approach should give you the desired output from your original SQL query but implemented in PySpark.

Problem Statement



DataFrame is given a dataset containing employee information with the following columns:

- Dept (Department name)
- Employee (Employee name)
- Salary (Employee's salary)

The dataset contains records for multiple employees across different departments. The objective is to calculate the minimum and maximum salaries for each department using Spark SQL. Additionally, you need to sort the result by the maximum salary in descending order.

Input Table Data

```
schdata ="""Dept, Employee,Salary
  HR, Alice, 50000,
  HR, Bob, 55000
  HR, Charlie, 52000
  Finance, David, 75000
  Finance, Eve, 70000
  Finance, Frank, 78000
  IT, Grace, 60000
  IT, Hank, 62000
  IT, Ivy, 61000
  """

df=
spark.read.format('csv').option('header',True).option('inferschema',True).load('/Filestore/tables/data.csv')
```

Expected Output

Dept	Min_Salary	Max_Salary
Finance	70000	78000
IT	60000	62000
HR	50000	55000

###Problem Statement: Department-Wise Minimum and Maximum Salary Calculation Using Spark SQL dataframe containing employee information with the following columns:

Dept (Department name)

Employee (Employee name)

Salary (Employee's salary)

The dataset contains records for multiple employees across different departments. The objective is to calculate the minimum and maximum salaries for each department using Spark SQL. Additionally, you need to sort the result by the maximum salary in descending order.

```
# Data as a list of tuples
data ="""Dept, Employee,Salary
    HR, Alice, 50000,
    HR, Bob, 55000
    HR, Charlie, 52000
    Finance, David, 75000
    Finance, Eve, 70000
    Finance, Frank, 78000
    IT, Grace, 60000
    IT, Hank, 62000
    IT, Ivy, 61000
    """

dbutils.fs.put('/Filestore/tables/data.csv',data,True)
```

```
Wrote 226 bytes.
Out[23]: True
```

```
# Create a DataFrame
df =
spark.read.format('csv').option('header',True).option('inferschema',Tr
ue).load('/Filestore/tables/data.csv')

df.display()

df.printSchema()
```

```
root
 |-- Dept: string (nullable = true)
 |--  Employee: string (nullable = true)
 |-- Salary: double (nullable = true)
```

```
from pyspark.sql.functions import max, min

# Calculate department-wise minimum and maximum salary and order by
Max_Salary
df_min_max_salary =
df.groupBy(df['Dept']).agg(min(df['Salary']).alias("Min_Salary"),max(d
```

```
f['Salary']).alias("Max_Salary"))

df_min_max_salary.display()
```

#Saprk SQL

```
df.createOrReplaceTempView("MinMaxSalaryView")

result = spark.sql("""
    SELECT Dept, MIN(Salary) AS Min_Salary, MAX(Salary) AS Max_Salary
    FROM MinMaxSalaryView
    GROUP BY Dept
    ORDER BY Max_Salary DESC
""")
result.display()
```

Problem Statement



Find Employees with Minimum and Maximum Salaries in Each Department Given a dataset containing employee names, department IDs, and their corresponding salaries, the goal is to determine the following for each department: The name of the employee with the minimum salary. The name of the employee with the maximum salary. Input: A table or dataset containing the following columns:

emp_name (String): Name of the employee.
dept_id (Integer): ID of the department the employee belongs to.
salary (Integer): Salary of the employee.

Input Table Data

```
# Define schema
schema = StructType([ StructField("emp_name", StringType(), True), StructField("dept_id", IntegerType(), True), StructField("salary", IntegerType(), True) ])

 # Create DataFrame with data
data = [ ("Siva", 1, 30000), ("Ravi", 2, 40000), ("Prasad", 1, 50000), ("Sai", 2, 20000), ("Anna", 2, 10000) ]

emps_df = spark.createDataFrame(data, schema)
```

Expected Output

dept_id	min_sal_empname	max_sal_empname
1	Siva	Prasad
2	Anna	Ravi

###Problem Statement: Find Employees with Minimum and Maximum Salaries in Each Department Given a dataset containing employee names, department IDs, and their corresponding salaries, the goal is to determine the following for each department:

The name of the employee with the minimum salary. The name of the employee with the maximum salary. Input: A table or dataset containing the following columns:

emp_name (String): Name of the employee.

dept_id (Integer): ID of the department the employee belongs to.

salary (Integer): Salary of the employee.

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType,
IntegerType
from pyspark.sql import functions as F
from pyspark.sql import Window

# Create Spark session
spark =
SparkSession.builder.appName("CreateTableExample").getOrCreate()

# Define schema
schema = StructType([
    StructField("emp_name", StringType(), True),
    StructField("dept_id", IntegerType(), True),
    StructField("salary", IntegerType(), True)
])

# Create DataFrame with data
data = [
    ("Siva", 1, 30000),
    ("Ravi", 2, 40000),
    ("Prasad", 1, 50000),
    ("Sai", 2, 20000),
    ("Anna", 2, 10000)
]

emps_df = spark.createDataFrame(data, schema)

# Show the DataFrame content
emps_df.display()


emps_df.createOrReplaceTempView('emps_wipro')
```

#Spark SQL

```sql
%sql
with t as (
  select
    dept_id,
    first_value(emp_name) over(
      partition by dept_id
      order by
        salary
    ) as min_sal_empname,
    last_value(emp_name) over(
      partition by dept_id
      order by
        salary rows between unbounded preceding
        and unbounded following
    ) as max_sal_empname
  from
    emps_wipro
)
select
  distinct dept_id,
  min_sal_empname,
  max_sal_empname
from
  t;
```

#Pyspark

```python
# Create window specification partitioned by dept_id
window_spec = Window.partitionBy("dept_id").orderBy("salary")

# Get employee with minimum salary in each department
min_sal_df = emps_df.withColumn("rank",
F.row_number().over(window_spec)) \
                    .filter(F.col("rank") == 1) \
                    .select("dept_id",
F.col("emp_name").alias("min_sal_empname"))

# Get employee with maximum salary in each department
max_sal_df = emps_df.withColumn("rank",
F.row_number().over(window_spec.orderBy(F.col("salary").desc()))) \
                    .filter(F.col("rank") == 1) \
                    .select("dept_id",
F.col("emp_name").alias("max_sal_empname"))

# Join the two DataFrames on dept_id
result_df = min_sal_df.join(max_sal_df, on="dept_id", how="inner")

# Show the result
result_df.display()
```

###Explanation: Window Specification: This is used to rank the employees by salary within each department.

Minimum Salary Employee: We use the window to find the employee with the lowest salary in each department.

Maximum Salary Employee: Similarly, we find the employee with the highest salary in each department.

Join: Finally, we join both DataFrames on the dept_id to get the desired result.

This will give you the output with dept_id, min_sal_empname, and max_sal_empname as required.

Problem Statement



You are given two datasets: one containing cricket players' statistics (player name, runs, 50s/100s) and another mapping country codes to full country names.
Your task is to:

1. Extract player names and country codes from the player column.
2. Split the 50s/100s column into separate values for 50s and 100s.
3. Join the players' dataset with the countries' dataset using the country code.
4. Calculate the sum of 50s and 100s for each player.
5. Filter to include only players with a sum greater than 90.
6. Output the player's name, country, runs, and the calculated sum, sorted by country.

The goal is to use PySpark to process the data and produce the desired output.

Input Table Data

```
# Define schema for players_df

schema = StructType([
  StructField("player", StringType(), True),
  StructField("runs", IntegerType(), True),
  StructField("50s/100s", StringType(), True)
])

# Create players_df DataFrame
data = [
  ("Sachin-IND", 18694, "93/49"),
  ("Ricky-AUS", 11274, "66/31"),
  ("Lara-WI", 10222, "45/21"),
  ("Rahul-IND", 10355, "95/11"),
  ("Jhonty-SA", 7051, "43/5"),
  ("Hayden-AUS", 8722, "67/19")
]
players_df = spark.createDataFrame(data, schema)

# Create countries_df DataFrame
data1 = [("IND", "India"), ("AUS", "Australia"), ("WI", "WestIndies"), ("SA",
"SouthAfrica")]
countries_df = spark.createDataFrame(data1, ["SRT", "country"])
```

```
+----------+----------+------+-----+
| playername|   country|  runs|  sum|
+---------+----------+-------+-------+
|    Ricky  |  Australia|  11274|   97  |
|   Sachin|     India   |  18694|  142|
|    Rahul |     India   |  10355|  106|
+----------+----------+------+-------+
```

#Problem Statement: You are given two datasets representing cricket players' statistics and their associated country information. Your task is to transform and analyze this data using PySpark.

Dataset 1: Players

This dataset contains information about cricket players, including their names, total runs scored, and their 50s and 100s in the format player-COUNTRY_CODE. Columns: player: The name of the player concatenated with a hyphen and the country code (e.g., Sachin-IND). runs: The total runs scored by the player. 50s/100s: The number of 50s and 100s scored by the player, separated by a forward slash (e.g., 93/49).

Dataset 2: Countries

This dataset maps the country code to the full country name. Columns: SRT: The country code (e.g., IND). country: The full name of the country (e.g., India).

Task: Extract Information:

Split the player column in the Players dataset into two separate fields: player_name: The name of the player. SRT: The country code. Split the 50s/100s column into two separate fields: runs_50s: The number of 50s. runs_100s: The number of 100s. Join the Datasets:

Perform an inner join between the transformed Players dataset and the Countries dataset using the SRT field to map country codes to full country names. Calculate the Sum:

Calculate the sum of 50s and 100s for each player. Filter:

Filter the results to include only those players whose sum of 50s and 100s is greater than 90. Sort:

Sort the filtered results by country name.

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType,
IntegerType

# Initialize Spark session
spark = SparkSession.builder.appName("CricketData").getOrCreate()

# Define schema for players_df
schema = StructType([
    StructField("player", StringType(), True),
    StructField("runs", IntegerType(), True),
    StructField("50s/100s", StringType(), True)
])

# Create players_df DataFrame
data = [
    ("Sachin-IND", 18694, "93/49"),
    ("Ricky-AUS", 11274, "66/31"),
```

```
    ("Lara-WI", 10222, "45/21"),
    ("Rahul-IND", 10355, "95/11"),
    ("Jhonty-SA", 7051, "43/5"),
    ("Hayden-AUS", 8722, "67/19")
]
players_df = spark.createDataFrame(data, schema)

# Create countries_df DataFrame
data1 = [("IND", "India"), ("AUS", "Australia"), ("WI", "WestIndies"),
("SA", "SouthAfrica")]
countries_df = spark.createDataFrame(data1, ["SRT", "country"])

# Show the DataFrames
players_df.display()
countries_df.display()

players_df.createOrReplaceTempView('players')
countries_df.createOrReplaceTempView('country')

# Execute the Spark SQL query
result_df = spark.sql("""
    WITH player AS (
        SELECT
            player,
            runs,
            SPLIT(player, '-')[0] AS player_name,
            SPLIT(player, '-')[1] AS SRT,
            SPLIT(`50s/100s`, '/')[0] AS runs_50s,
            SPLIT(`50s/100s`, '/')[1] AS runs_100s
        FROM players
    )
    SELECT
        p.player_name AS playername,
        c.country,
        p.runs,
        CAST(p.runs_50s AS INT) + CAST(p.runs_100s AS INT) AS sum
    FROM player p
    INNER JOIN country c
    ON p.SRT = c.SRT
    WHERE CAST(p.runs_50s AS INT) + CAST(p.runs_100s AS INT) > 90
    ORDER BY c.country
""")

# Show the result
result_df.display()

from pyspark.sql.functions import col, expr, split

# Split the player column into player_name and SRT using the '-'
delimiter
```

```
players_transformed_df = players_df.withColumn(
    "player_name", split(col("player"), "-").getItem(0)
).withColumn("SRT", split(col("player"), "-").getItem(1))

# Split the 50s/100s column into runs_50s and runs_100s using the '/'
delimiter
players_transformed_df = (
    players_transformed_df.withColumn(
        "runs_50s", split(col("50s/100s"), "/").getItem(0).cast("int")
    )
    .withColumn("runs_100s", split(col("50s/100s"),
"/").getItem(1).cast("int"))
    .withColumn("total_50s_100s", col("runs_50s") + col("runs_100s"))
)

# Join with countries_df on SRT
result_df = (
    players_transformed_df.join(
        countries_df, players_transformed_df["SRT"] ==
countries_df["SRT"]
    )
    .select(
        col("player_name").alias("playername"),
        col("country"),
        col("runs"),
        col("total_50s_100s").alias("sum"),
    )
    .where(col("total_50s_100s") > 90)
    .orderBy("country")
)

# Show the result
result_df.display()
```

Explanation of Changes:

Splitting Columns: Instead of trying to use substring, we use the split() function in PySpark to split the player and 50s/100s columns into separate parts.

split(col("player"), "-").getItem(0): Extracts the part of the string before the hyphen (player name). split(col("player"), "-").getItem(1): Extracts the part of the string after the hyphen (country code). Similarly, split(col("50s/100s"), "/").getItem(0) and split(col("50s/100s"), "/").getItem(1) are used to extract the number of 50s and 100s, and they are cast to int. Column Operations: Operations like adding columns are done using the standard PySpark column functions, which are designed to work on DataFrame columns.

This approach avoids the "Column is not iterable" error and produces the correct output.

Problem Statement



Given a table tbl_maxval with columns col1, col2, and col3, write a SQL query to find the record(s) where the maximum value occurs in either col2 or col3.

Input Table Data

```
# Define schema for tbl_maxval

schema = StructType([
  StructField("col1", StringType(), True),
  StructField("col2", IntegerType(), True),
  StructField("col3", IntegerType(), True)
])

# Insert records into tbl_maxval
data = [
  ('a', 10, 20),
  ('b', 50, 30)
]

# Create a DataFrame from the data and schema
df = spark.createDataFrame(data, schema)
```

Output Table

col1	max_val
a	20
b	50

#problem statement:

Given a table tbl_maxval with columns col1, col2, and col3, write a SQL query to find the record(s) where the maximum value occurs in either col2 or col3.

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType,
IntegerType
from pyspark.sql.functions import col, max

# Initialize Spark session
spark = SparkSession.builder \
    .appName("Create Table tbl_maxval") \
    .getOrCreate()

# Define the schema for the tbl_maxval table
schema = StructType([
    StructField("col1", StringType(), True),
    StructField("col2", IntegerType(), True),
    StructField("col3", IntegerType(), True)
])

# Insert records into tbl_maxval
data = [
    ('a', 10, 20),
    ('b', 50, 30)
]

# Create a DataFrame from the data and schema
df = spark.createDataFrame(data, schema)

# Show the DataFrame contents
df.display()



# Optionally, register the DataFrame as a temporary view to run SQL
queries
df.createOrReplaceTempView("tbl_maxval")

# Example SQL query (optional)
spark.sql("SELECT * FROM tbl_maxval").display()

%sql
select col1, col2
from tbl_maxval
union all
select col1, col3
from tbl_maxval
```

```
%sql
with cte as (
select col1, col2
from tbl_maxval
union all
select col1, col3
from tbl_maxval
)
select col1, max(col2) as maxval
from cte
group by col1;
```

```
cte = df.select("col1", "col2").unionAll(df.select("col1",
col("col3").alias("col2")))

# Perform GROUP BY and calculate the maximum value
result = cte.groupBy("col1").agg(max("col2").alias("maxval"))

# Show the results
result.display()
```

#Explanation: Here is the step-by-step explanation of the solution:

Steps: Find the maximum value from col2 and col3.

Subquery to find the maximum value:

The (MAX(col2), Max(col3)) function is used to find the maximum value across both columns. This subquery returns the highest value between col2 and col3. Filter records:

The outer SELECT query filters the records where either col2 or col3 equals the maximum value found in the subquery.

Problem Statement



ou have a dataset of daily exchange rates for various currencies stored in a table exchange_rates with the following structure:
currency_code (e.g., 'USD', 'EUR'): The code for the currency.
date: The date of the exchange rate.
currency_exchange_rate: The exchange rate for the currency on that date.
Your task is to derive the first and last exchange rates for each currency on a monthly basis. Specifically, you need to: Create a new identifier, currency_yearmonth, which combines the currency code, year, and month of each record. For each currency_yearmonth, find: The first exchange rate of the month (first_day_rate). The last exchange rate of the month (last_day_rate). Return distinct records with the fields currency_yearmonth, first_day_rate, and last_day_rate.

Input Table Data

```
# Define schema for tbl_maxval

data = [
  ("USD", date(2024, 6, 1), Decimal("1.20")),
  ("USD", date(2024, 6, 2), Decimal("1.21")),
  ("USD", date(2024, 6, 3), Decimal("1.22")),
  ("USD", date(2024, 6, 4), Decimal("1.23")),
  ("USD", date(2024, 7, 1), Decimal("1.25")),
  ("USD", date(2024, 7, 2), Decimal("1.26")),
  ("USD", date(2024, 7, 3), Decimal("1.27")),
  ("EUR", date(2024, 6, 1), Decimal("1.40")),
  ("EUR", date(2024, 6, 2), Decimal("1.41")),
  ("EUR", date(2024, 6, 3), Decimal("1.42")),
  ("EUR", date(2024, 6, 4), Decimal("1.43")),
  ("EUR", date(2024, 7, 1), Decimal("1.45")),
  ("EUR", date(2024, 7, 2), Decimal("1.46")),
  ("EUR", date(2024, 7, 3), Decimal("1.47")),
]

# Create DataFrame
df = spark.createDataFrame(data, schema)
```

Output Table

currency_yearmonth	first_day_rate	last_day_rate
EUR-2024-6	1.40	1.43
EUR-2024-7	1.45	1.47
USD-2024-6	1.20	1.23
USD-2024-7	1.25	1.27

Problem Statement:

You have a dataset of daily exchange rates for various currencies stored in a table exchange_rates with the following structure:

currency_code (e.g., 'USD', 'EUR'): The code for the currency.

date: The date of the exchange rate.

currency_exchange_rate: The exchange rate for the currency on that date.

Your task is to derive the first and last exchange rates for each currency on a monthly basis. Specifically, you need to:

Create a new identifier, currency_yearmonth, which combines the currency code, year, and month of each record. For each currency_yearmonth, find: The first exchange rate of the month (first_day_rate). The last exchange rate of the month (last_day_rate). Return distinct records with the fields currency_yearmonth, first_day_rate, and last_day_rate.

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType,
DateType, DecimalType
from decimal import Decimal
from datetime import date

# Initialize Spark session
spark =
SparkSession.builder.appName("ExchangeRatesExample").getOrCreate()

# Define schema for exchange_rates table
schema = StructType(
    [
        StructField("currency_code", StringType(), True),
        StructField("date", DateType(), True),
        StructField("currency_exchange_rate", DecimalType(10, 2),
True),
    ]
)

# Define data to insert
data = [
    ("USD", date(2024, 6, 1), Decimal("1.20")),
    ("USD", date(2024, 6, 2), Decimal("1.21")),
    ("USD", date(2024, 6, 3), Decimal("1.22")),
    ("USD", date(2024, 6, 4), Decimal("1.23")),
    ("USD", date(2024, 7, 1), Decimal("1.25")),
    ("USD", date(2024, 7, 2), Decimal("1.26")),
    ("USD", date(2024, 7, 3), Decimal("1.27")),
    ("EUR", date(2024, 6, 1), Decimal("1.40")),
    ("EUR", date(2024, 6, 2), Decimal("1.41")),
    ("EUR", date(2024, 6, 3), Decimal("1.42")),
```

```
    ("EUR", date(2024, 6, 4), Decimal("1.43")),
    ("EUR", date(2024, 7, 1), Decimal("1.45")),
    ("EUR", date(2024, 7, 2), Decimal("1.46")),
    ("EUR", date(2024, 7, 3), Decimal("1.47")),
]

# Create DataFrame
df = spark.createDataFrame(data, schema)

# Write DataFrame to a temporary table or to a specific location
df.write.mode("overwrite").saveAsTable("exchange_rates")

# Optionally, show the data
df.display()

df.createOrReplaceTempView("exchange_rates")

%sql
WITH cte AS(
  SELECT
    currency_code,
    CONCAT(currency_code, '-', YEAR(date), '-', MONTH(date)) AS
currency_yearmonth,
    currency_exchange_rate
  FROM
    exchange_rates
)
SELECT
  DISTINCT currency_yearmonth,
  FIRST_VALUE(currency_exchange_rate) OVER(
    PARTITION BY currency_yearmonth
    ORDER BY
      currency_yearmonth
  ) AS first_day_rate,
  LAST_VALUE(currency_exchange_rate) OVER(
    PARTITION BY currency_yearmonth
    ORDER BY
      currency_yearmonth ROWS BETWEEN UNBOUNDED PRECEDING
      AND UNBOUNDED FOLLOWING
  ) AS last_day_rate
FROM
  cte;

from pyspark.sql import Window
from pyspark.sql.functions import col, first, last, concat, lit, year,
month

# Define a CTE by selecting relevant columns and constructing
'currency_yearmonth'
cte_df = df.select(
```

```
    col("currency_code"),
    concat(
        col("currency_code"), lit("-"), year(col("date")), lit("-"),
month(col("date"))
    ).alias("currency_yearmonth"),
    col("currency_exchange_rate"),
    col("date"),
)

# Define window specification for partitioning by 'currency_yearmonth'
and ordering by 'date'
window_spec = Window.partitionBy("currency_yearmonth").orderBy("date")

# Calculate first and last exchange rates using window functions
result_df = (
    cte_df.withColumn(
        "first_day_rate",
first("currency_exchange_rate").over(window_spec)
    )
    .withColumn(
        "last_day_rate",
        last("currency_exchange_rate").over(
            window_spec.rowsBetween(
                Window.unboundedPreceding, Window.unboundedFollowing
            )
        ),
    )
    .select("currency_yearmonth", "first_day_rate", "last_day_rate")
    .distinct()
)

# Show the result
result_df.display()
```

Explanation:

CTE equivalent (cte_df): This creates a DataFrame with currency_code, currency_yearmonth, currency_exchange_rate, and date. Window specification: We define a window partitioned by currency_yearmonth and ordered by date.

Window functions: We use first() and last() to calculate the first and last exchange rates within each currency_yearmonth. The rowsBetween(Window.unboundedPreceding, Window.unboundedFollowing) ensures that the correct first and last values are selected for each partition. Distinct results: We select distinct combinations of currency_yearmonth, first_day_rate, and last_day_rate. Display: Finally, the resulting DataFrame is displayed.

Problem Statement



Given a travel route dataset containing information about origin and destination cities, you need to identify the travel routes that occur only once in the dataset. Specifically:

1. Input: A dataset with two columns:
    - origin: The city from which the travel originates.
    - destination: The city to which the travel is headed.
2. Task:
    - Perform a self-join on the dataset to find matching routes where the origin of one route matches the destination of another.
    - For each origin and destination pair, calculate the count of how many times that pair appears in the dataset.
    - Filter out the pairs that appear more than once, leaving only those that appear once or not at all.
    - Return the list of routes (origin-destination pairs) that appear at most once in the dataset, ordered by the destination city.

Input Table Data

```
# Define schema for tbl_maxval

schema = StructType([
  StructField("Origin", StringType(), True),
  StructField("Destination", StringType(), True)
])

# Define the data
data = [
  ("Bangalore", "Chennai"),
  ("Chennai", "Bangalore"),
  ("Pune", "Chennai"),
  ("Delhi", "Pune")
]

# Create a DataFrame
routes_df = spark.createDataFrame(data, schema)
```

Output Table

ORIGIN	DESTINATION
Bangalore	Chennai
Pune	Chennai
Delhi	Pune

Problem statement:

Given a travel route dataset containing information about origin and destination cities, you need to identify the travel routes that occur only once in the dataset. Specifically:

Input: A dataset with two columns:

origin: The city from which the travel originates. destination: The city to which the travel is headed. Task:

Perform a self-join on the dataset to find matching routes where the origin of one route matches the destination of another. For each origin and destination pair, calculate the count of how many times that pair appears in the dataset. Filter out the pairs that appear more than once, leaving only those that appear once or not at all. Return the list of routes (origin-destination pairs) that appear at most once in the dataset, ordered by the destination city.

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType
from pyspark.sql import functions as F

# Create a Spark session
spark = SparkSession.builder.appName("RoutesDataFrame").getOrCreate()

# Define the schema for the DataFrame
schema = StructType([
    StructField("Origin", StringType(), True),
    StructField("Destination", StringType(), True)
])

# Define the data
data = [
    ("Bangalore", "Chennai"),
    ("Chennai", "Bangalore"),
    ("Pune", "Chennai"),
    ("Delhi", "Pune")
]

# Create a DataFrame
routes_df = spark.createDataFrame(data, schema)

# Show the DataFrame
routes_df.display()

routes_df.createOrReplaceTempView('travel')

# Perform a self join to match t1.origin with t2.destination
joined_df = routes_df.alias("t1").join(
    routes_df.alias("t2"),
    F.col("t1.origin") == F.col("t2.destination"),
    how="left"
)
```

```
# Group by t1.origin and t1.destination and calculate the count
cte_df = joined_df.groupBy("t1.origin",
"t1.destination").agg(F.count("*").alias("cnt"))

# Filter rows where cnt <= 1 and order by destination
result_df = cte_df.filter(F.col("cnt") <= 1).orderBy("t1.destination")

# Show the result
result_df.select("origin", "destination").display()
```

Explanation:

Sample Data: We create a sample dataset that simulates your travel table.

Self Left Join: The travel_df DataFrame is joined to itself, using a left join on the condition t1.origin = t2.destination.

Group By and Aggregate: We group the results by t1.origin and t1.destination, and compute the count of rows.

Filter and Order: We filter the DataFrame to only include rows where the count (cnt) is less than or equal to 1, and then sort the results by the destination column.

Select and Display: Finally, we select the origin and destination columns and display the result.

```
%sql
with cte as(
  select
    t1.origin,
    t1.destination,
    count(*) as cnt
  from
    travel t1
    left join travel t2 on t1.origin = t2.destination
  group by
    t1.origin,
    t1.destination
)
select
  origin,
  destination
from
  cte
where
  cnt <= 1
order by
  destination
```

Explanation:

Sample Data: This simulates your travel table. Self Join: The DataFrame travel_df is joined to itself based on the condition t1.origin = t2.destination.

Group By: We group by t1.origin and t1.destination and compute the count of occurrences.

Filter and Order: We filter the DataFrame to include only rows where the count (cnt) is less than or equal to 1, then order the result by destination.

This PySpark code replicates the behavior of your SQL CTE query using DataFrame operations. Let me know if you need any adjustments!

Problem Statement



To find the 2nd Wednesday of the current month and display it in Spark SQL with the columns DayDate, Weekday, and RowNum, you can use the following SQL query. This query will generate all dates for the current month, identify Wednesdays, rank them, and select the 2nd Wednesday...

Input Table Data

```
# Define schema for tbl_maxval

start_date = datetime(2024, 1, 1)
end_date = datetime(2024, 12, 31)
date_list = [
  (start_date + timedelta(days=x)).strftime("%Y-%m-%d")
  for x in range((end_date - start_date).days + 1)
]

# Create a DataFrame with the list of dates
date_df = spark.createDataFrame([(date,) for date in date_list], ["DayDate"])
date_df_with_weekday = date_df.withColumn(
  "Weekday", date_format(col("DayDate"), "EEEE")
)
```

Output Table

DayDate	Weekday	RowNum
2024-01-10	Wednesday	2

Problem Statement:

To find the 2nd Wednesday of the current month and display it in Spark SQL with the columns DayDate, Weekday, and RowNum, you can use the following SQL query. This query will generate all dates for the current month, identify Wednesdays, rank them, and select the 2nd Wednesday...

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, date_format, expr, row_number
from pyspark.sql.window import Window
from datetime import datetime, timedelta

# Initialize Spark session
spark = SparkSession.builder \
    .appName("Find 2nd Wednesday") \
    .getOrCreate()

# Define the current date and the first and last dates of the current
month
now = datetime.now()
start_date = datetime(now.year, now.month, 1)
end_date = (start_date + timedelta(days=31)).replace(day=1) -
timedelta(days=1)

# Generate a list of dates for the current month
date_list = [(start_date + timedelta(days=x)).strftime('%Y-%m-%d') for
x in range((end_date - start_date).days + 1)]

# Create DataFrame from the list
date_df = spark.createDataFrame([(date,) for date in date_list],
["DayDate"])

# Add a column for the day of the week as a string
date_df = date_df.withColumn("Weekday", date_format(col("DayDate"),
"EEEE"))
# Show the DataFrame to verify
date_df.display()

# Add a column for the day of the week number (1 = Monday, ..., 7 =
Sunday)
date_df = date_df.withColumn("WeekDayNum",
expr("dayofweek(to_date(DayDate, 'yyyy-MM-dd'))"))

# Filter for Wednesdays (WeekDayNum = 4, since dayofweek function
returns 1 for Sunday)
wednesdays_df = date_df.filter(col("WeekDayNum") == 4)  # 4 represents
Wednesday

# Rank Wednesdays and select the 2nd one
window_spec = Window.orderBy("DayDate")
```

```
wednesdays_ranked_df = wednesdays_df.withColumn("RowNum",
row_number().over(window_spec))

# Select the 2nd Wednesday
second_wednesday_df = wednesdays_ranked_df.filter(col("RowNum") == 2)

# Show the result
second_wednesday_df.select("DayDate", "Weekday", "RowNum").display()

date_df_with_weekday.createOrReplaceTempView("DayDate")

%sql
-- Create a temporary view with all dates for the current month
-- Create a temporary view with all dates for the current month
WITH DateSeries AS (
    -- Generate all dates for the current month
    SELECT
        DATE_ADD(DATE_TRUNC('month', CURRENT_DATE()), seq) AS DayDate
    FROM (
        SELECT posexplode(sequence(0, DAY(LAST_DAY(CURRENT_DATE())) -
1)) AS (seq, day)
    )
),
Wednesdays AS (
    -- Filter out only Wednesdays
    SELECT
        DayDate,
        CASE DAYOFWEEK(DayDate)
            WHEN 1 THEN 'Sunday'
            WHEN 2 THEN 'Monday'
            WHEN 3 THEN 'Tuesday'
            WHEN 4 THEN 'Wednesday'
            WHEN 5 THEN 'Thursday'
            WHEN 6 THEN 'Friday'
            WHEN 7 THEN 'Saturday'
        END AS Weekday
    FROM DateSeries
    WHERE DAYOFWEEK(DayDate) = 4 -- 4 represents Wednesday (where
Sunday = 1)
),
RankedWednesdays AS (
    -- Rank the Wednesdays
    SELECT
        DayDate,
        Weekday,
        ROW_NUMBER() OVER (ORDER BY DayDate) AS RowNum
    FROM Wednesdays
)
-- Select the 2nd Wednesday
SELECT
```

```
    DayDate,
    Weekday,
    RowNum
FROM RankedWednesdays
WHERE RowNum = 2;
```

Explanation:

Generate Dates:

DATE_TRUNC('month', CURRENT_DATE()): Gets the first day of the current month. sequence(0, DAY(LAST_DAY(CURRENT_DATE())) - 1): Creates a sequence of integers representing days from 0 to the last day of the month. DATE_ADD: Adds these days to the start date to generate all dates for the current month. Identify Wednesdays:

DAYOFWEEK: Determines the weekday number. This function returns 1 for Sunday, 2 for Monday, ..., 7 for Saturday. Thus, 4 represents Wednesday. CASE: Converts the weekday number to the day name. Rank and Select:

ROW_NUMBER(): Ranks the Wednesdays in ascending order of date. WHERE RowNum = 2: Filters to select the 2nd Wednesday. Notes Day of the Week: Adjust the CASE expression and DAYOFWEEK logic based on your Spark SQL configuration if needed. Temporary View: If you run this in a notebook or interactive SQL environment, ensure you have the necessary permissions to create and query temporary views. This query will return the 2nd Wednesday of the current month, showing the DayDate, the day of the week as Weekday, and the RowNum.

Input Table Data

```
# Define schema for tbl_maxval

products_schema = StructType([
  StructField("pid", IntegerType(), True),
  StructField("pname", StringType(), True),
  StructField("price", IntegerType(), True)
])

# Define schema for transactions table
transactions_schema = StructType([
  StructField("pid", IntegerType(), True),
  StructField("sold_date", DateType(), True),
  StructField("qty", IntegerType(), True),
  StructField("amount", IntegerType(), True)
])

# Create DataFrame for products
products_data = [
  (1, 'A', 1000),
  (2, 'B', 400),
  (3, 'C', 500)
]
products_df                    =                    spark.createDataFrame(products_data,
schema=products_schema)

# Create DataFrame for transactions
transactions_data = [
  (1, datetime.strptime('2024-02-01', '%Y-%m-%d').date(), 2, 2000),
  (1, datetime.strptime('2024-03-01', '%Y-%m-%d').date(), 4, 4000),
  (1, datetime.strptime('2024-03-15', '%Y-%m-%d').date(), 2, 2000),
  (3, datetime.strptime('2024-04-24', '%Y-%m-%d').date(), 3, 1500),
  (3, datetime.strptime('2024-05-16', '%Y-%m-%d').date(), 5, 2500)
]
```

Problem Statement



Generate a monthly sales report for all products in the year 2024. The report should include the total sales for each product by month, ensuring that even months with no sales data are represented with a total sales value of zero.

...

Output Table

pid	pname	years	month	total_sales
1	A	2024	1	0
1	A	2024	2	2000
1	A	2024	3	6000
1	A	2024	4	0
1	A	2024	5	0
1	A	2024	6	0
1	A	2024	7	0
1	A	2024	8	0
1	A	2024	9	0
1	A	2024	10	0
1	A	2024	11	0
1	A	2024	12	0
2	B	2024	1	0
2	B	2024	2	0
2	B	2024	3	0
2	B	2024	4	0
2	B	2024	5	0
2	B	2024	6	0
2	B	2024	7	0
2	B	2024	8	0
2	B	2024	9	0
2	B	2024	10	0
2	B	2024	11	0
2	B	2024	12	0
3	C	2024	1	0
3	C	2024	2	0
3	C	2024	3	0
3	C	2024	4	1500
3	C	2024	5	2500
3	C	2024	6	0
3	C	2024	7	0
3	C	2024	8	0
3	C	2024	9	0
3	C	2024	10	0
3	C	2024	11	0
3	C	2024	12	0

Problem Statement:

Generate a monthly sales report for all products in the year 2024. The report should include the total sales for each product by month, ensuring that even months with no sales data are represented with a total sales value of zero.

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, IntegerType,
StringType, DateType
from datetime import datetime

# Initialize Spark session
spark = SparkSession.builder \
    .appName("ProductTransactions") \
    .getOrCreate()

# Define schema for products table
products_schema = StructType([
    StructField("pid", IntegerType(), True),
    StructField("pname", StringType(), True),
    StructField("price", IntegerType(), True)
])

# Define schema for transactions table
transactions_schema = StructType([
    StructField("pid", IntegerType(), True),
    StructField("sold_date", DateType(), True),
    StructField("qty", IntegerType(), True),
    StructField("amount", IntegerType(), True)
])

# Create DataFrame for products
products_data = [
    (1, 'A', 1000),
    (2, 'B', 400),
    (3, 'C', 500)
]
products_df = spark.createDataFrame(products_data,
schema=products_schema)

# Create DataFrame for transactions
transactions_data = [
    (1, datetime.strptime('2024-02-01', '%Y-%m-%d').date(), 2, 2000),
    (1, datetime.strptime('2024-03-01', '%Y-%m-%d').date(), 4, 4000),
    (1, datetime.strptime('2024-03-15', '%Y-%m-%d').date(), 2, 2000),
    (3, datetime.strptime('2024-04-24', '%Y-%m-%d').date(), 3, 1500),
    (3, datetime.strptime('2024-05-16', '%Y-%m-%d').date(), 5, 2500)
]
transactions_df = spark.createDataFrame(transactions_data,
schema=transactions_schema)
```

```
# Show the dataframes
print("Products:")
products_df.display()

print("Transactions:")
transactions_df.display()

# Further processing or analysis can be done using Spark SQL or
DataFrame operations
# For example, joining products and transactions to calculate total
sales per product
joined_df = transactions_df.join(products_df, on='pid', how='inner')
joined_df.display()
```

Products:

Transactions:

```
from pyspark.sql import functions as F

# Assuming `products_df` and `transactions_df` are already created

# Step 1: Create the base DataFrame with `pid`, `pname`, `years`, and
`month` initialized to 1.
initial_df = products_df.alias("p").join(transactions_df.alias("t"),
F.col("p.pid") == F.col("t.pid"), 'left') \
    .select(F.col("p.pid").alias("pid"),
F.col("p.pname").alias("pname"),
            F.when(F.col("t.sold_date").isNotNull(),
F.year("t.sold_date")).otherwise(2024).alias("years"),
            F.lit(1).alias("month")).distinct()

# Step 2: Create DataFrame for all months (1 to 12) using a loop.
months_df = initial_df
for i in range(2, 13):  # Loop from month 2 to 12
    new_month_df = initial_df.withColumn("month", F.lit(i))
    months_df = months_df.union(new_month_df)

# Step 3: Join with `transactions_df` again to calculate total_sales
for each `pid`, `pname`, `years`, and `month`.
final_df = months_df.alias("r").join(transactions_df.alias("t"),
                                     (F.col("r.pid") ==
F.col("t.pid")) &
                                     (F.month("t.sold_date") ==
F.col("r.month")),
                                     'left')

# Step 4: Perform group by to aggregate `total_sales`
final_df = final_df.groupBy("r.pid", "r.pname", "r.years", "r.month")
\
```

```
    .agg(F.sum("t.amount").alias("total_sales"))

# Step 5: Replace `NULL` values in `total_sales` with 0.
final_df = final_df.withColumn("total_sales",
F.coalesce(F.col("total_sales"), F.lit(0)))

# Step 6: Order the final result.
final_df = final_df.orderBy("r.pid", "r.pname", "r.years", "r.month")

# Display the final result
final_df.display()


products_df.createOrReplaceTempView('products')
transactions_df.createOrReplaceTempView('transactions')

%sql
WITH initial_data AS (
    -- Step 1: Create the base DataFrame with `pid`, `pname`, `years`,
and `month` initialized to 1.
    SELECT
        p.pid,
        p.pname,
        COALESCE(YEAR(t.sold_date), 2024) AS years,
        1 AS month
    FROM products p
    LEFT JOIN transactions t ON p.pid = t.pid
    GROUP BY p.pid, p.pname, COALESCE(YEAR(t.sold_date), 2024)
),

-- Step 2: Create DataFrame for all months (1 to 12)
all_months_data AS (
    SELECT pid, pname, years, 1 AS month FROM initial_data
    UNION ALL SELECT pid, pname, years, 2 AS month FROM initial_data
    UNION ALL SELECT pid, pname, years, 3 AS month FROM initial_data
    UNION ALL SELECT pid, pname, years, 4 AS month FROM initial_data
    UNION ALL SELECT pid, pname, years, 5 AS month FROM initial_data
    UNION ALL SELECT pid, pname, years, 6 AS month FROM initial_data
    UNION ALL SELECT pid, pname, years, 7 AS month FROM initial_data
    UNION ALL SELECT pid, pname, years, 8 AS month FROM initial_data
    UNION ALL SELECT pid, pname, years, 9 AS month FROM initial_data
    UNION ALL SELECT pid, pname, years, 10 AS month FROM initial_data
    UNION ALL SELECT pid, pname, years, 11 AS month FROM initial_data
    UNION ALL SELECT pid, pname, years, 12 AS month FROM initial_data
),

-- Step 3: Join with `transactions` again to calculate total_sales
joined_data AS (
    SELECT
        am.pid,
```

```
        am.pname,
        am.years,
        am.month,
        SUM(t.amount) AS total_sales
    FROM all_months_data am
    LEFT JOIN transactions t
        ON am.pid = t.pid
        AND MONTH(t.sold_date) = am.month
        AND YEAR(t.sold_date) = am.years
    GROUP BY am.pid, am.pname, am.years, am.month
)

-- Step 4: Replace NULL values in `total_sales` with 0 and order the
result
SELECT
    pid,
    pname,
    years,
    month,
    COALESCE(total_sales, 0) AS total_sales
FROM joined_data
ORDER BY pid, pname, years, month;
```

Explanation:

1. Initial DataFrame (initial_df): This corresponds to the first part of the recursive CTE, where we select distinct products and set the initial month to
2. Manual Recursion (Loop): Instead of using recursion (which PySpark doesn't support), we generate the next months (2 to 12) by looping and unioning the DataFrame for each month.
3. Join and Aggregation: We use a left join to match each product's month with transaction data and aggregate the amount column to calculate total_sales. Handling NULL Values: The coalesce() function replaces NULL values in total_sales with 0.
4. Ordering: The result is ordered by pid, pname, years, and month, mimicking the final ORDER BY in the SQL query.

Problem Statement



We have a table named employees_tbl containing employee details, including their employee ID (eid), name (ename), and gender (gender). You want to compute the following:

The total number of employees in the table.

The count of employees grouped by gender (i.e., how many males and females are there).

The percentage of employees by gender, rounded to the nearest whole number.

This calculation needs to be done using PySpark, replicating the functionality of SQL common table expressions (CTEs) without using built-in SQL functions in PySpark.

Input Table Data

```
# Define schema for tbl_maxval

schema = StructType(
  [
    StructField("eid", IntegerType(), True),
    StructField("ename", StringType(), True),
    StructField("gender", StringType(), True),
  ]
)

# Data to insert into the table
data = [
  (1, "John Doe", "Male"),
  (2, "Jane Smith", "Female"),
  (3, "Michael Johnson", "Male"),
  (4, "Emily Davis", "Female"),
  (5, "Robert Brown", "Male"),
  (6, "Sophia Wilson", "Female"),
  (7, "David Lee", "Male"),
  (8, "Emma White", "Female"),
  (9, "James Taylor", "Male"),
  (10, "William Clark", "Male"),
]
```

Output Table

gender	gender_count	gender_perc
Male	6	60
Female	4	40

Problem Statement:

We have a table named employees_tbl containing employee details, including their employee ID (eid), name (ename), and gender (gender). You want to compute the following:

The total number of employees in the table.

The count of employees grouped by gender (i.e., how many males and females are there).

The percentage of employees by gender, rounded to the nearest whole number.

This calculation needs to be done using PySpark, replicating the functionality of SQL common table expressions (CTEs) without using built-in SQL functions in PySpark.

```
# Import necessary PySpark modules
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, IntegerType,
StringType

# Initialize Spark session
spark = SparkSession.builder.appName(
    "Create and Insert into Employees Table"
).getOrCreate()

# Define schema for the table
schema = StructType(
    [
        StructField("eid", IntegerType(), True),
        StructField("ename", StringType(), True),
        StructField("gender", StringType(), True),
    ]
)

# Data to insert into the table
data = [
    (1, "John Doe", "Male"),
    (2, "Jane Smith", "Female"),
    (3, "Michael Johnson", "Male"),
    (4, "Emily Davis", "Female"),
    (5, "Robert Brown", "Male"),
    (6, "Sophia Wilson", "Female"),
    (7, "David Lee", "Male"),
    (8, "Emma White", "Female"),
    (9, "James Taylor", "Male"),
    (10, "William Clark", "Male"),
]

# Create DataFrame from data and schema
df = spark.createDataFrame(data, schema)
```

```
# Show the contents of the table (for verification)
df.display()

# Save DataFrame to a temporary table (can be registered as a SQL
table if needed)
df.createOrReplaceTempView("employees_tbl")

%sql
WITH CTE1 as(
  SELECT
    count(*) as total_emp
  FROM
    employees_tbl
),
CTE2 AS(
  SELECT
    gender,
    count(*) as gender_count
  FROM
    employees_tbl
  GROUP BY
    gender
)
SELECT
  CTE2.*,
  ROUND((CTE2.gender_count / CTE1.total_emp) * 100, 0) as gender_perc
FROM
  CTE2,
  CTE1;

from pyspark.sql import functions as F

# Step 1: Calculate total employee count (CTE1 equivalent)
total_emp_df = df.agg(F.count("*").alias("total_emp"))

# Step 2: Calculate gender count grouped by gender (CTE2 equivalent)
gender_count_df =
df.groupBy("gender").agg(F.count("*").alias("gender_count"))

# Step 3: Perform a cross join between total_emp_df and
gender_count_df
result_df = gender_count_df.crossJoin(total_emp_df)

# Step 4: Calculate gender percentage
result_df = result_df.withColumn(
    "gender_perc", F.round((F.col("gender_count") /
F.col("total_emp")) * 100, 0)
)
```

Problem Statement



Here are given a dataset of product production records, which includes information about the production date (dt), brand name (brand), model name (model), and production cost (production_cost). The goal is to calculate the total production cost for each brand on a specific date using a window function that partitions the data by dt (date) and brand, and orders the data by production_cost.

For each record, you need to compute the sum of the production costs within the same dt and brand group. This sum should be consistent for all rows that belong to the same partition.

Input Table Data

```
# Define schema for tbl_maxval
schema = StructType([
  StructField("dt", DateType(), True),
  StructField("brand", StringType(), True),
  StructField("model", StringType(), True),
  StructField("production_cost", IntegerType(), True)
])

# Data for the table
data = [
  ('2023-12-01', 'A', 'A1', 1000),
  ('2023-12-01', 'A', 'A2', 1300),
  ('2023-12-01', 'B', 'B1', 800),
  ('2023-12-02', 'A', 'A1', 1800),
  ('2023-12-02', 'B', 'B1', 900),
  ('2023-12-10', 'A', 'A1', 1400),
  ('2023-12-10', 'A', 'A1', 1200),
  ('2023-12-10', 'C', 'C1', 2500)
]

# Convert the data to Rows
rows = [Row(datetime.strptime(row[0], '%Y-%m-%d'),
row[1], row[2], row[3]) for row in data]
```

Output Table

dt	brand	model	production_cost	sum_cost
2023-12-01	A	A1	1000	2300
2023-12-01	A	A2	1300	2300
2023-12-01	B	B1	800	800
2023-12-02	A	A1	1800	1800
2023-12-02	B	B1	900	900
2023-12-10	A	A1	1200	2600
2023-12-10	A	A1	1400	2600
2023-12-10	C	C1	2500	2500

Problem Statement:

Here are given a dataset of product production records, which includes information about the production date (dt), brand name (brand), model name (model), and production cost (production_cost). The goal is to calculate the total production cost for each brand on a specific date using a window function that partitions the data by dt (date) and brand, and orders the data by production_cost.

For each record, you need to compute the sum of the production costs within the same dt and brand group. This sum should be consistent for all rows that belong to the same partition.

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType,
IntegerType, DateType
from pyspark.sql import Row
from datetime import datetime

# Initialize Spark session
spark =
SparkSession.builder.appName("CreateTableAndInsertRecords").getOrCreat
e()

# Define schema
schema = StructType([
    StructField("dt", DateType(), True),
    StructField("brand", StringType(), True),
    StructField("model", StringType(), True),
    StructField("production_cost", IntegerType(), True)
])

# Data for the table
data = [
    ('2023-12-01', 'A', 'A1', 1000),
    ('2023-12-01', 'A', 'A2', 1300),
    ('2023-12-01', 'B', 'B1', 800),
    ('2023-12-02', 'A', 'A1', 1800),
    ('2023-12-02', 'B', 'B1', 900),
    ('2023-12-10', 'A', 'A1', 1400),
    ('2023-12-10', 'A', 'A1', 1200),
    ('2023-12-10', 'C', 'C1', 2500)
]

# Convert the data to Rows
rows = [Row(datetime.strptime(row[0], '%Y-%m-%d'), row[1], row[2],
row[3]) for row in data]

# Create DataFrame
df = spark.createDataFrame(rows, schema)
```

```
# Show the DataFrame
df.display()

# Register the DataFrame as a temporary table
df.createOrReplaceTempView("prd_tbl")

%sql
select
  *,
  sum(production_cost) over (
    partition by dt,
    brand
    order by
      production_cost rows between unbounded preceding
      and unbounded following
  ) as sum_cost
from
  prd_tbl

from pyspark.sql.window import Window
from pyspark.sql.functions import sum

# Define the window specification
window_spec = Window.partitionBy("dt",
"brand").orderBy("production_cost").rowsBetween(Window.unboundedPreced
ing, Window.unboundedFollowing)

# Calculate the sum of production_cost over the window
df_with_sum_cost = df.withColumn("sum_cost",
sum("production_cost").over(window_spec))

# Show the result
df_with_sum_cost.display()
```

Explanation:

Window.partitionBy("dt", "brand"): This creates partitions based on the dt and brand columns, similar to PARTITION BY dt, brand in SQL.

orderBy("production_cost"): This orders the rows within each partition by production_cost.

rowsBetween(Window.unboundedPreceding, Window.unboundedFollowing): This specifies that the sum should be calculated over all rows within the partition (equivalent to ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING in SQL).

sum("production_cost").over(window_spec): This computes the sum of production_cost over the defined window.

This code calculates the rolling sum of production_cost for each dt and brand group, ordered by production_cost, and adds the result as a new column called sum_cost.

Input Table Data

```
# Define schema for the DataFrame
schema = StructType(
  [
    StructField("empid", IntegerType(), True),
    StructField("dept", StringType(), True),
    StructField("salary", IntegerType(), True),
    StructField("date", StringType(), True), # Assuming date as
StringType for now
  ]
)

# Input data
data = [
  (100, "IT", 100, "2024-05-12"),
  (200, "IT", 100, "2024-06-12"),
  (100, "FIN", 400, "2024-07-12"),
  (300, "FIN", 500, "2024-07-12"),
  (300, "FIN", 1543, "2024-07-12"),
  (300, "FIN", 1500, "2024-07-12"),
]

# Create DataFrame
df = spark.createDataFrame(data, schema=schema)
```

Problem Statement



You have a DataFrame containing employee details, including employee IDs (empid), departments (dept), salaries (salary), and dates (date). Your goal is to identify and retrieve the details of employees who appear exactly once in the DataFrame, i.e., employees whose empid is unique in the dataset.

Output Table

EMPID	DEPT	SALARY	DATE
200	IT	100	2024-06-12

Problem Statement:

Here we have a DataFrame containing employee details, including employee IDs (empid), departments (dept), salaries (salary), and dates (date). Your goal is to identify and retrieve the details of employees who appear exactly once in the DataFrame, i.e., employees whose empid is unique in the dataset.

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, IntegerType,
StringType, DateType

# Initialize Spark session
spark = SparkSession.builder.appName("Create DataFrame
Example").getOrCreate()

# Define schema for the DataFrame
schema = StructType(
    [
        StructField("empid", IntegerType(), True),
        StructField("dept", StringType(), True),
        StructField("salary", IntegerType(), True),
        StructField("date", StringType(), True),  # Assuming date as
StringType for now
    ]
)

# Input data
data = [
    (100, "IT", 100, "2024-05-12"),
    (200, "IT", 100, "2024-06-12"),
    (100, "FIN", 400, "2024-07-12"),
    (300, "FIN", 500, "2024-07-12"),
    (300, "FIN", 1543, "2024-07-12"),
    (300, "FIN", 1500, "2024-07-12"),
]

# Create DataFrame
df = spark.createDataFrame(data, schema=schema)

# Show DataFrame
df.display()

df.createOrReplaceTempView("test")

%sql
with cte as (
  select
    *,
    row_number() over(
      partition by empid
```

```
      order by
        empid
    ) ranks
  from
    test
),
cte1 as(
  select
    empid,
    count(ranks) as varun
  from
    cte
  group by
    empid
  having
    count(ranks) = 1
)
select
  *
from
  test
where
  empid in (
    select
      empid
    from
      cte1
  )

from pyspark.sql.functions import col, count

# Step 1: Group by 'empid' and count occurrences
emp_count_df = df.groupBy("empid").agg(count("empid").alias("count"))

# Step 2: Filter for empid with count of 1
single_occurrence_df = emp_count_df.filter(col("count") ==
1).select("empid")

# Step 3: Join with original DataFrame to get full details
non_repeated_emp_details_df = df.join(single_occurrence_df,
on="empid", how="inner")

# Show the result
non_repeated_emp_details_df.display()

Explanation:

To solve this problem, follow these steps:

Group by empid:
```

```
Objective: Determine how many times each empid appears in the dataset.
Approach: Use the groupBy method on the empid column and aggregate
using count. This will give you a new DataFrame where each row
represents an empid and the number of occurrences of that empid.
Filter for empid with Count of 1:

Objective: Identify empid values that appear exactly once.
Approach: Use the filter method to select rows from the grouped
DataFrame where the count is 1. This will give you a DataFrame
containing only those empid values which are unique in the dataset.
Join with Original DataFrame:

Objective: Retrieve the full details of the employees identified in
the previous step.
Approach: Perform an inner join between the original DataFrame and the
filtered DataFrame (containing unique empid values). This join will
return only the rows from the original DataFrame where the empid is in
the filtered list of unique empid values.
```

Problem Statement



Here are working with a PySpark DataFrame that contains several columns, and some of the cells in these columns contain null values. Your goal is to calculate the total number of null values for each column in the DataFrame. Specifically, you want to:
Count the number of null values in each column. Display the result as a new DataFrame, where each column represents the count of null values for the corresponding column in the original DataFrame. This task mirrors an SQL query that counts null values using CASE statements and the SUM() function for each column.

Input Table Data

```
# Define schema for the DataFrame
data = [(1, None, "ab"), (2, 10, None), (None, None, "cd")]
columns = ["col1", "col2", "col3"]


# Create DataFrame
df = spark.createDataFrame(data, schema=schema)
```

Output Table

col1_null_count	col2_null_count	col3_null_count
1	2	1

Problem Statement:

Here are working with a PySpark DataFrame that contains several columns, and some of the cells in these columns contain null values. Your goal is to calculate the total number of null values for each column in the DataFrame. Specifically, you want to:

Count the number of null values in each column. Display the result as a new DataFrame, where each column represents the count of null values for the corresponding column in the original DataFrame. This task mirrors an SQL query that counts null values using CASE statements and the SUM() function for each column.

```
from pyspark.sql import SparkSession

# Create a Spark session
spark = SparkSession.builder.appName("ExampleApp").getOrCreate()

# Sample data
data = [(1, None, "ab"), (2, 10, None), (None, None, "cd")]
columns = ["col1", "col2", "col3"]

# Create DataFrame
df = spark.createDataFrame(data, columns)

# Show the DataFrame
df.display()

df.createOrReplaceTempView("emp")

%sql
select
  sum(
    case
      when col1 is null then 1
      else 0
    end
  ) as col1,
  sum(
    case
      when col2 is null then 1
      else 0
    end
  ) as col2,
  sum(
    case
      when col3 is null then 1
      else 0
    end
  ) as col3
from
  emp;
```

```
from pyspark.sql import functions as F

# Count the number of nulls in each column
df_null_count = df.select(
    F.sum(F.when(F.col("col1").isNull(),
1).otherwise(0)).alias("col1_null_count"),
    F.sum(F.when(F.col("col2").isNull(),
1).otherwise(0)).alias("col2_null_count"),
    F.sum(F.when(F.col("col3").isNull(),
1).otherwise(0)).alias("col3_null_count")
)

# Show the result
df_null_count.display()
```

Explanation:

F.when(F.col("col1").isNull(), 1).otherwise(0): This creates a column that returns 1 if the value is null, otherwise 0. F.sum(...): This sums up the 1s to count the null values. .alias("col1_null_count"): This renames the output columns to indicate the null count for each column.

Problem Statement



You are given a dataset of travel records where each record includes a customer ID, a start location, and an end location. The dataset is represented as follows:
customer: An identifier for the customer. start_location: The location from where the customer starts their journey. end_location: The location where the customer ends their journey. Your task is to identify and list the following for each customer:
Start Locations: Locations where the customer started a journey but did not end any journey. End Locations: Locations where the customer ended a journey but did not start any journey. You need to produce a result that matches the following criteria:
For each customer, find start locations that do not appear as end locations in any records for that customer. For each customer, find end locations that do not appear as start locations in any records for that customer. Return a combined result showing the customer, their unique start location, and the corresponding unique end location where the conditions above are met.

Input Table Data

```
# Define schema for the DataFrame
data = [
  ("c1", "New York", "Lima"),
  ("c1", "London", "New York"),
  ("c1", "Lima", "Sao Paulo"),
  ("c1", "Sao Paulo", "New Delhi"),
  ("c2", "Mumbai", "Hyderabad"),
  ("c2", "Surat", "Pune"),
  ("c2", "Hyderabad", "Surat"),
  ("c3", "Kochi", "Kurnool"),
  ("c3", "Lucknow", "Agra"),
  ("c3", "Agra", "Jaipur"),
  ("c3", "Jaipur", "Kochi"),
]

# Define the schema
schema = "customer string, start_location string, end_location string"

# Create the DataFrame
df = spark.createDataFrame(data=data, schema=schema)
```

Output Table

customer	start	end
c1	London	New Delhi
c2	Mumbai	Pune
c3	Lucknow	Kurnool

Problem Statement:

You are given a dataset of travel records where each record includes a customer ID, a start location, and an end location. The dataset is represented as follows:

customer: An identifier for the customer. start_location: The location from where the customer starts their journey. end_location: The location where the customer ends their journey. Your task is to identify and list the following for each customer:

Start Locations: Locations where the customer started a journey but did not end any journey. End Locations: Locations where the customer ended a journey but did not start any journey. You need to produce a result that matches the following criteria:

For each customer, find start locations that do not appear as end locations in any records for that customer. For each customer, find end locations that do not appear as start locations in any records for that customer. Return a combined result showing the customer, their unique start location, and the corresponding unique end location where the conditions above are met.

```
from pyspark.sql import SparkSession

# Create a Spark session
spark = SparkSession.builder.appName("DataFrameExample").getOrCreate()

# Sample data
data = [
    ("c1", "New York", "Lima"),
    ("c1", "London", "New York"),
    ("c1", "Lima", "Sao Paulo"),
    ("c1", "Sao Paulo", "New Delhi"),
    ("c2", "Mumbai", "Hyderabad"),
    ("c2", "Surat", "Pune"),
    ("c2", "Hyderabad", "Surat"),
    ("c3", "Kochi", "Kurnool"),
    ("c3", "Lucknow", "Agra"),
    ("c3", "Agra", "Jaipur"),
    ("c3", "Jaipur", "Kochi"),
]

# Define the schema
schema = "customer string, start_location string, end_location string"

# Create the DataFrame
df = spark.createDataFrame(data=data, schema=schema)

# Show the DataFrame
df.display()

from pyspark.sql.functions import collect_list, udf
from pyspark.sql.types import StringType
```

```
def loc(x, y):
    a = [i for i in x if i not in y]
    return a[0]


loc_udf = udf(loc, StringType())
df1 = df.groupBy("customer").agg(
    collect_list("start_location").alias("start_list"),
    collect_list("end_location").alias("end_list"),
)
display(df1)
df2 = (
    df1.withColumn("start", loc_udf(df1.start_list, df1.end_list))
    .withColumn("end", loc_udf(df1.end_list, df1.start_list))
    .drop(*("start_list", "end_list"))
)
df2.display()

df.createOrReplaceTempView('travel_data')

# SQL query to get the results
query = """
WITH t1 AS (
  SELECT
    customer,
    start_location AS start
  FROM
    travel_data
  WHERE
    start_location NOT IN (
      SELECT
        end_location
      FROM
        travel_data
    )
),
t2 AS (
  SELECT
    customer,
    end_location AS end
  FROM
    travel_data
  WHERE
    end_location NOT IN (
      SELECT
        start_location
      FROM
        travel_data
    )
)
```

```
SELECT
  t1.customer,
  t1.start,
  t2.end
FROM
  t2
  JOIN t1 ON t2.customer = t1.customer
"""

# Execute the SQL query
result_df = spark.sql(query)

# Show the result
result_df.display()
```

Explanation:

customer: Represents the customer ID.

start_location: Represents the starting location of a journey.

end_location: Represents the ending location of a journey.